ZAPPER

MANUAL · ESP32-C6 · FIRMWARE 1.12.2

A lamp that blinks as soon as someone sends you a zap over nostr. It has its own nostr account, listens on the relays itself, and is set up entirely in the browser. This sheet belongs with the lamp. Keep it with the lamp.

There are two versions. The **Zapper REV A** is the finished board, with a USB-A socket for the lamp, two buttons, a buzzer and a power meter. The **lamp01** is the first prototype, an ESP32-C6-Zero with a separate MOSFET. Where they differ, this manual says so.

Contents

1. The board and its connections
2. First-time setup
3. What the status light says
4. The buttons on the board
5. Zapping for everyone
6. How it reacts to a zap
7. Power and load (REV A)
8. Goal, celebrations, alerts and webhook
9. When something does not work
10. Zap card and payments without a zap
11. Bitcoin on-chain (optional)
12. Being found everywhere
13. Taking the lamp to another home or network
14. More than one lamp
15. Maintenance
16. Technical data
17. About ZAPPER

The board and its connections

Zapper REV A

PART	WHAT IT IS FOR
USB-C	Power for the board *and* the lamp. Use a **5 V / 3 A** adapter and a short cable. To a PC, the same socket is used for flashing firmware.
USB-A	The lamp. The output is switched and protected by a TPS2559: on a short or too much current it switches off by itself.
SETUP	Wi-Fi and resetting: 5 or 10 seconds. A short press writes the state to the event log.
ACTION	A test zap: the lamp blinks, the status light flashes gold and the buzzer beeps.
BOOT, RESET	Small buttons for flashing over USB. You do not need them in daily use.
Status light	An RGB LED that shows how things stand. See *What the status light says*.
Buzzer	Beeps on a zap, on the buttons and on a fault. Can be made quieter or silent.

The board has no on/off switch. To keep the lamp on or off regardless of zaps, choose **Always on** or **Always off** on the Lamp tab; that choice stays after a restart. See *How it reacts to a zap*.

Unplug the lamp from the USB-A socket before you connect the board to a PC: the PC port cannot also power a lamp.

lamp01 (the prototype)

CONNECTION	GOES TO
`GPIO4`	The **SIG** input of the MOSFET. This is how it switches and dims the lamp.
`GND`	The ground of the MOSFET, also tied to the ground of the 5 V supply. Without that shared ground the MOSFET does not switch.
`5 V lamp`	Through the MOSFET, on the external 5 V supply, not on the ESP32 itself.
`USB-C`	Power for the board, and the cable you flash firmware with.

The status light and the BOOT button are already on the board (GPIO8 and GPIO9). You do not wire anything to those.

Power (lamp01)

Use **one external 5 V supply** for both the lamp and the board. The USB-C port is then only needed for programming, or for reading the serial log.

```
external 5 V supply
     │
     ├── 5V ──── lamp (+)
     │            lamp (-) ── MOSFET  drain
     │                        MOSFET  source ── GND
     ├── 5V ──── 5V pin of the board
     └── GND ─── GND of the board and the MOSFET

                 GPIO4 ──────── MOSFET SIG
```

That way the lamp current does not flow through the ESP32 board. Do not power the lamp through the USB-C port of the board: that current then goes through the connector and across the traces of the PCB, and that small board is not made for it.

> **Never two supplies at once**
> If the external 5 V is on the `5V` pin, disconnect it before you plug in the USB-C cable. Otherwise two supplies sit on the same rail. The same rule as when programming: disconnect the external supply first.

First-time setup

These instructions use the English labels. The web interface opens in Dutch; switch it with **EN** at the top of the page.

1. Plug in the power: the adapter in the USB-C socket (REV A), or the 5 V supply (lamp01). The status light breathes **light blue**: it is waiting for you.

2. On your phone, find the Wi-Fi network `zapper-xxxx-setup` and join it. Each lamp has its own `xxxx`, so you can tell them apart. The setup screen opens on its own; if it does not, go to `http://192.168.4.1`.

3. Choose your own Wi-Fi, enter the password, give the lamp a **name** (for example `zapper01`) and choose a password for the lamp. Press **Connect**; the lamp restarts.

4. Go to `http://zapper01.local` (with the name you chose) and log in. If you left the password empty, it is `Bitcoin21`. Change it after the first login under **Settings**; until you do, **Getting started** reminds you that with the default password anyone on your network can get in, including to the lamp's key.

5. At the top is **Getting started**: a list of what is still left. Open the **Zapping** tab. Under **1. Lightning address** enter your address (for example `you@example.com`), press **Check this address**, and then under **2. Announce on nostr** press **Publish profile to relays**.

Anyone can now zap the lamp. On that same tab is a QR code of its npub: scanning it with a nostr client opens the lamp's profile directly, ready to zap. The sats land in your own wallet. The lamp stores nothing and cannot reach them.

> **Publish again if you change the relays**
> When it publishes, the lamp sends two things: its profile with your lightning address, and the list of relays it listens on. That second item tells the zapper's nostr client where the receipt must go. Without it, the client has to guess, and a zap can pass the lamp by.

What is required, and what is optional

KIND	WHAT
Required	Wi-Fi, a lightning address (**Zapping, step 1**) and announcing on nostr (**Zapping, step 2**). After that, zapping works from every nostr app.
Optional	Bitcoin on-chain (a new block, a payment to your address or xpub), zapping with any lightning wallet, even without nostr (**Extras**), the zap card, the thank-you note, profile and picture, quiet hours, the status light, Bluetooth, a webhook, and more lamps.
Fine by default	How the lamp reacts, what it listens for, the relays and the key.
Recommended	Your own password instead of Bitcoin21.

In the web interface this is the label at the top right of each card.

Setup over Bluetooth

If your phone will not stay on a Wi-Fi network without internet, or the lamp is somewhere the setup network barely reaches, you can do it over Bluetooth too. The ESP32 has that on board, and the lamp turns it on as soon as it is in setup mode.

1. Install the free app **ESP BLE Provisioning** from Espressif (App Store and Google Play).
2. Make sure the lamp is in setup mode: brand new, or hold the button for 5 seconds. The light breathes light blue.
3. Open the app, choose **Provision device** and then **I don't have a QR code**.
4. Pick `PROV_<name>` from the list, for example `PROV_zapper01`.
5. For the PIN (Proof of Possession) enter the **lamp password**: `Bitcoin21` by default.
6. Choose your Wi-Fi network, enter the password and press connect. The lamp restarts and gets to work.

In the web interface, **Settings → Set up over Bluetooth** shows a QR code that fills in the name and the PIN in one go; in the app you can then choose *I have a QR code*. Take a photo of it before you move house.

> **Bluetooth is not always on**
> Only during setup mode, and for twenty minutes at most. Once the lamp is on your Wi-Fi the radio stays off. In normal operation it transmits nothing. You can turn it off completely under **Settings → Set up over Bluetooth**. The PIN travels encrypted over the air (x25519), never in the clear.

Prefer zaps to your own account?

Then the lamp does not need its own lightning address. Under **Settings → What it listens for**, set it to *Zaps to another npub* and enter your own npub. People then zap you as usual, and the lamp watches along. For most people that is the easiest setup.

You can follow up to four accounts at once, one per line. Besides an npub you may also enter a nostr address such as `you@example.com`; the lamp looks that up itself. As soon as it sees the profile, the name appears beside it, so you can tell you have the right one.

> #### "This connection is not secure"
> That message from your browser is correct: the traffic between you and the lamp is unencrypted. Your password itself does not travel in the clear. The browser first computes a one-time code with it. The rest of the traffic can be read by someone on your Wi-Fi. For a device on your own network that is usual. If you do want it encrypted, reach it through Tailscale.

> #### Mind the `http://`
> The lamp speaks ordinary HTTP. On `https://zapper01.local` nothing happens, because it is not listening there. Make sure a bookmark or your browser's automatic HTTPS preference does not turn it into `https`.

What the status light says

The small RGB LED on the board tells you at a glance how things stand.

Green, steady	Connected and listening. This is how it should sit.
Orange, steady	Wi-Fi is fine, but no relay is reachable. Usually the internet is down.
Red, steady	Wi-Fi is gone. It keeps trying on its own. You do not need to do anything.
Blue, pulsing	Connecting.
Light blue, breathing	Setup mode: connect your phone to `<name>-setup`.
Green and blue, alternating	Your phone is on the setup network.
Blue, five quick flashes	The Wi-Fi was just cleared. It comes back in setup mode.
Gold flash	A zap came in. The big lamp blinks along with it.
Purple, pulsing	Firmware is being updated. Do not remove power now.
Orange, blooming three times	A new bitcoin block, if you switched that on under *Extras*.
Rainbow	The fundraising goal is reached.
Red, three quick flashes	REV A: the protection switched the lamp output off. Also when you hold SETUP for 10 seconds: let go now to clear everything.
Short orange blink every 4 s	REV A: the lamp is heavy for the board. On top of the normal colour. See *Power and load*.
Double red blink every 2 s	REV A: the lamp is too heavy. Choose a lamp that draws less.
White, steady	Starting up, for a second or two after power on.
White, blinking	You pressed *Which one is this?* (the crosshair) in the web interface.

Everything dimmed? Quiet hours are active. How bright the light is by day and at night is set under **Lamp → Status light on the board**.

The buttons on the board

The buttons are the emergency path: they work even when you can no longer reach the web interface. On REV A that is **SETUP**, on lamp01 the **BOOT** button.

`short`	REV A: a beep, and the state (Wi-Fi, IP address, relays) goes into the event log. lamp01: a test flash of the big lamp as well.
`5 seconds`	Forget the Wi-Fi. The light flashes blue five times (on REV A with a beep, so you know you can let go) and it comes back in setup mode, ready for a new network. Your nostr key and all settings stay.
`10 seconds`	Clear everything: Wi-Fi, password, settings and the nostr key. The light flashes red (on REV A with a low tone); let go then. The lamp gets a new npub.

REV A also has **ACTION**. A short press is a test zap: the lamp blinks, the light flashes gold and the buzzer beeps, without any sats. Useful to check the lamp without a phone. While a celebration tune plays, a short press on ACTION stops it instead.

> **Do not hold BOOT while powering on**
> BOOT is also the programming button. If you hold it while power is applied, the chip starts in programming mode and does nothing else. Power off and on again clears that. On REV A the separate BOOT button is for this; SETUP and ACTION are safe.

Zapping for everyone

Not everyone has a nostr app. That is why the lamp has its own **zap page** that works with *any* lightning wallet. The overview in the web interface shows a QR code for it. Let guests scan that.

1. The guest opens `http://zapper01.local/zap` and chooses an amount, for example 21 sats.
2. A QR code appears. They scan it with their lightning wallet: Wallet of Satoshi, Phoenix, Strike, Alby, whatever they have.
3. They pay. The lamp blinks and the page says *Thank you!* (in Dutch it says *Bedankt!*).

How does that work without nostr? The lamp has its own key and creates the zap request itself. The guest only pays the invoice. No password is needed, the page only works on your own network, and the sats go to your lightning address.

Someone who does have a nostr app can also just scan the npub QR code and zap from that app. It is on the zap page and on the Nostr tab.

How it reacts to a zap

The lamp is simply on. When a zap comes in, it blinks for a while and then stays on again. In the web interface you set two things: **how long** it blinks and **how fast**.

SETTING	WHAT IT DOES
How long	1 to 60 seconds of blinking per zap
How fast	0.2 to 10 times per second. Above 5 it is more of a flicker than a blink
Longer for bigger amounts	Multiplies the time by the number of digits: 21 sats = 2x as long, 5000 sats = 4x as long
At most	An upper limit, so a huge zap does not go on for minutes
Ignore below	Zaps smaller than this amount leave it alone

An example: 6 seconds at 2.5 times per second, with *longer for bigger amounts* on. A zap of 21 sats then blinks for 12 seconds, about 30 times on and off. If another zap comes in while it is blinking, the pattern is extended.

You can also leave the lamp off when idle, so it turns on only for a zap. That is under *When idle* on the Lamp tab.

On **REV A** the lamp is switched on and off, not dimmed: the protected output does not allow dimming. A zap there is a blink pattern. Brightness, glow and fading are hidden in the web interface on that board.

Automatic, Always on, Always off

On the Lamp tab you choose the mode. **Automatic** is normal: on at rest, blinking on a zap. **Always on** and **Always off** are manual: the lamp ignores zaps until you set it back. The choice survives a restart and a power cut. The event log says so at startup: `mode kept from before the restart`.

Power and load (REV A)

The REV A board measures the current and the voltage on its USB-A output. On the **Overview**, the card **Power** shows the current on a gauge, with a verdict underneath. Handy when you put a different lamp on the board.

VERDICT	WHEN	WHAT IT MEANS
Safe	up to 1.6 A	Fine with any decent adapter.
Careful	up to 2.3 A, or below 4.75 V	It works, but use a good 5 V / 3 A adapter and a short cable, and feel whether things get warm.
Too heavy	above 2.3 A, or below 4.5 V	Choose a lamp that draws less. Around 2.5 A (the white tick on the gauge) the protection switches the output off.

A low voltage counts too: it points to a weak adapter or a long, thin cable rather than to the lamp. The verdict is taken while current flows, so a lamp that is off keeps its last verdict.

Next to the gauge: volts, watts, the peak current, the lowest voltage under load, the energy used, and a graph of the last half hour. **Reset peak and energy** starts those counts again. A step up to orange or red is written to the event log, under `power`.

Trying a new lamp

1. Plug the lamp into the USB-A socket.
2. On the Lamp tab, set it to **Always on** for half a minute.
3. Look at the gauge and the verdict.
4. Set it back to **Automatic**, or it will not blink on a zap.

The status light warns too

A short orange blink every four seconds means heavy; a double red blink every two seconds means too heavy. If the protection switches the output off, the light flashes red three times and the buzzer warns. The board then tries again on its own. With *Alerts on your phone* you also get a message.

Is the current reading right?

Every board is a few percent off. To correct it:

1. Put a multimeter, on the A range, in series with the lamp.
2. Set the lamp to **Always on**.
3. Open *Power → Calibrate the current*, enter what the meter reads and save.

The correction is stored on this board only, and does not travel with the settings file. A value more than 30 % off is refused: then something else is wrong. **Back to default** undoes it.

Goal, celebrations, alerts and webhook

Fundraising goal

Handy for an event or a fundraiser. Under **Extras → Fundraising goal** set a goal in sats, for example 21,000, and what it is for. Everything that comes in from that moment counts: zaps, payments through your linked wallet, and on-chain payments.

- Progress shows on the overview, above the *Zap stack*, and on the guest page, so visitors see how far along it is.
- On lamp01 the lamp glows softly at rest and gets brighter as the goal gets closer.
- At the goal there is a little party: the lamp blinks for fifteen seconds, the status light runs a rainbow and, on REV A, the buzzer plays this lamp's tune. With the thank-you note on, a note goes out on nostr too.
- **Start over** sets the goal counter back to zero. The normal counters stay.

Whether the tune plays at the goal is set under **Tune at the goal**. It also plays with the buzzer on *No zap beep*. Only *Silent* and quiet hours keep it quiet.

Celebrations: a party at every milestone

Under **Extras → Celebrations and tunes**, switch **Milestones** on and choose the step, for example 21,000 sats. Every time the total passes another step, the lamp celebrates: a rainbow on the status light, the lamp blinks and the buzzer plays this lamp's tune. Counting starts when you switch it on, and the count survives a restart. **Count again from 0** starts over; the Zap stack stays as it is.

- One big zap past several milestones gives one celebration, not several.
- If a milestone comes in while a tune plays, it follows straight after.
- A short press on **ACTION** stops the tune, and any that were waiting.

A tune for each lamp

On REV A you choose the lamp's celebration tune on the same card, right below the sound on a zap; the play button lets you hear it and the stop button ends it. Give each lamp its own, so you can hear which one is celebrating.

KIND	TUNES
Classics	*De âlde Friezen* (the default, the refrain), Can-can, William Tell galop, Radetzky March, Rondo alla Turca, The Entertainer, Eine kleine Nachtmusik, Ode an die Freude
Bitcoin	Charge, Block found, To the moon, 21 million, Stack sats, Pump it Up
Your own	Can be added to the firmware when it is built; they appear in the list by themselves.

Alerts on your phone

The lamp can send you a message when something needs attention:

- the output protection stepped in (REV A);
- the lamp is too heavy for the board (REV A);
- the fundraising goal is reached;
- the lamp restarted after a crash or a voltage dip.

It works directly with the free app **ntfy**:

1. Install ntfy on your phone and subscribe to a topic with a name nobody will guess, such as `zapper-a8f3k2`. Anyone who knows the name can read along.
2. Under *Extras → Alerts on your phone*, enter `https://ntfy.sh/zapper-a8f3k2` and save.
3. Press **Test**. A message arrives within seconds. If nothing comes, the event log says why.

A fault or overload warning is sent at most once every ten minutes, so a lamp that keeps tripping does not flood your phone. Any service that accepts a POST with text works too.

When a lamp drops out

From version 1.12 the lamps keep an eye on each other. Every five minutes each lamp checks whether the other ZAPPERs it has seen still answer. Has one not answered for ten minutes (power out, Wi-Fi gone), you get one alert: *zapper03 has not been reachable for 10 minutes*, and another one when it is back. Only one lamp sends it, the one with the lowest address that still works, so five lamps do not send five messages. A lamp also tells you itself when it has been on Wi-Fi for ten minutes without reaching any relay, because then zaps do not come in. Set up alerts on at least one lamp, preferably all of them.

Webhook

Under *Extras → Forward to something else* you enter an address that receives a short message on every zap: the lamp name, the amount, the zapper and their comment. Use it to trigger something in n8n, Home Assistant or your own script. At startup the lamp also sends its IP address, handy when `.local` does not work on your phone. The alerts above go there too, as JSON with `"gebeurtenis":"melding"`.

When something does not work

Look below for **what you see**, not for what you think the cause is. The causes are listed per symptom from most likely to least likely.

I cannot open the lamp in the browser

CHECK	WHAT YOU DO
Is there `https` in the address bar?	Change it to `http`. The lamp does not listen on https, and many browsers fill that in on their own. If it works after that, update your bookmark.
Are you using an Android phone?	Android does not know `.local` names. Use the IP address; you find it in your router under connected devices, or have the lamp send it to you (see *Webhook*).
What colour is the light?	Red or light blue means it is not on your Wi-Fi. See the two points below.
Are you on the same network?	A guest network or another Wi-Fi band can be isolated from your ordinary network. Many routers block traffic between the guest network and the home network.
Did you change the name?	Then the address changed with it. The name is on the Settings tab; if you no longer know it, you will find the lamp in your router as `zapper-…` or under the name you gave it.

The light is red

Red means: *no Wi-Fi*. It keeps trying on its own, so often you do not need to do anything.

SITUATION	WHAT YOU DO
Just after a power cut	Nothing. The lamp is up before your router and waits until the router is there. Within a few minutes it turns green on its own.
Wi-Fi password changed	Hold the button on the board for **5 seconds** and set the Wi-Fi up again.
New router, same network name	That usually works on its own. If not: button for 5 seconds, set it up again.
The lamp is too far from the router	Move it closer or use a repeater. Below about −80 dBm it gets shaky; the signal strength is at the top of the web interface.
Stays red and you changed nothing	Remove power for ten seconds. If it still does not come back, set the Wi-Fi up again with the button.

The light is orange

Wi-Fi works, but the lamp cannot reach a single relay. That almost always means the **internet** is down, not your Wi-Fi.

- Does the internet work on your phone over that same Wi-Fi? If not, it is your connection, not the lamp.
- If it stays orange while the internet works, look under *Settings → Nostr → What it listens for* and check that working addresses are still there. Proven: `relay.damus.io`, `nos.lol`, `relay.primal.net`.
- A relay that speaks only TLS 1.3 does not work with this chip. `relay.nostr.band` is an example.
- If it stays orange for a quarter of an hour after it was connected before, the lamp restarts itself. That is normal behaviour, not a fault.

No zaps come in

CHECK	WHAT YOU DO
Is *Getting started* fully ticked?	That list on the overview says exactly what is still missing. It disappears only when everything is done.
Has the profile been published?	Without that, nobody knows your npub exists or where the sats should go. Nostr tab, button *Publish profile*.
Does the lightning address work?	Press *Check this address*. A typo is silent: a zap simply never arrives.
Did you change the relays?	Publish again. The zapper app uses your published relay list to decide where the receipt must go.
Are you zapping the right account?	If the lamp is set to *Zaps to the lamp itself*, you must zap the **lamp's** npub, not your own account.
Does it work with 1 sat?	Open `http://<name>.local/zap` and pay yourself 1 sat. If that works, the problem is on the zapper's side.

The zap arrives, but the lamp does not blink

Look in the **event log** on the overview; it records what happened for each zap. The log is written in English. Match the line you see.

WHAT THE EVENT LOG SAYS	MEANING
`sats: below the threshold of …`	The amount is smaller than your minimum. Under *Lamp → Ignore zaps smaller than*.
`sats during quiet hours, stayed quiet`	Quiet hours are active. Turn them off by setting the start hour and the end hour to the same value.
`sats received, did not flash: lamp is set to Always on` or `Always off`	Manual control. Set the mode back to *Automatic* on the Lamp tab. The mode stays after a restart or power cut, like a switch.
`rejected: not from our own lightning service`	The receipt does not come from the service behind your address. A real payment cannot do that; it is a fake zap, or you changed the lightning address without publishing again.
`rejected: signature does not match`	Someone tampered with the receipt. Rightly ignored.

The lamp itself does not turn on

First see whether it is the software or the wiring: on *Lamp*, set it to **Always on**. If it stays dark, it is the wiring.

- Is the external 5 V supply connected, and is its ground tied to the GND of the board? Without a shared ground the MOSFET does not switch, even when the signal is correct.
- Does the light on the board flash red three times? Then the protection switched the output off: too much current, or too hot. Unplug the lamp and look for a short, or use a lamp that draws less.
- Then set the mode back to *Automatic*, or it will not blink on a zap.

I have lost my password

Hold the button on the board for **10 seconds**. The light blinks red; then let go. Everything is cleared: Wi-Fi, password, settings and the nostr key. The password is then `Bitcoin21` again.

> **Your npub goes with it**
> The lamp creates a new key and therefore gets a new npub. Anyone who saved the old one then zaps an account nobody reads any more. Save your nsec (Nostr tab, *Show secret key*) if you want to be able to restore that npub.

The lamp restarts on its own

That is usually intended:

- A quarter of an hour with no relay at all while Wi-Fi does work. Something is stuck, and a restart clears it.
- The main loop is stuck (watchdog, after 60 seconds). Rare; it is not in the event log, because the chip restarts at once.
- Unstable power. A charger that is too weak, or a long thin cable, causes voltage dips, especially when the lamp switches on. Try another supply.

Under **System**, *Last restart* says why it restarted: power on, a software restart, the watchdog, a crash or a voltage dip (brownout). The last three can also reach your phone; see *Alerts on your phone*.

The setup network does not appear on my phone

- It is called `<name>-setup`. On a new lamp that is for example `zapper-a1b2-setup`. Is the light breathing light blue? Then it is transmitting.
- If the light is red, it is not in setup mode: it is still trying to join a known network. Hold the button for 5 seconds.
- If the screen does not open on its own after you connect, go to `http://192.168.4.1`. Phones see that this network has no internet and sometimes fall back to mobile data; turn mobile data off for a moment.
- If it really will not work, use Bluetooth: the app **ESP BLE Provisioning**, device `PROV_<name>`, PIN = the lamp password. See *Setup over Bluetooth*.

The colours of the light are wrong

Go to *Lamp → Status light on the board* and press **Colour test**. The LED runs through red, green, blue, light blue, orange, purple, gold and white, a second and a half each, and the page names each colour as it shows. If it shows red while everything works, the colour order is wrong: swap it if the test does not match. If you see nothing at all, try another GPIO pin, or set the brightness above 0.

On the Zapper REV A board the pins are fixed. There you set the colour balance instead: if white pulls towards green or blue, or orange looks yellow, turn green or blue down until it is right.

The lamp beeps on every zap

A lamp with a buzzer (the Zapper REV A board) plays a short tune on every zap. Under *Lamp → Status light on the board → Buzzer* you choose:

- **Everything:** on a zap, the buttons and warnings.
- **No zap beep:** the zap is silent, but the buttons still confirm. Holding SETUP still beeps after 5 and after 10 seconds, so you know when to let go, and a fault on the output still sounds a warning.
- **Silent:** nothing at all.

Under *Extras → Celebrations and tunes*, **Sound on a zap** chooses what plays on every zap: the short beep (the default) or one of the tunes, such as the Can-can or *Stack sats*. It plays only with the buzzer on *Everything*, and a short press on ACTION stops it.

During quiet hours a zap never beeps, whatever you choose here. The tune for the goal and the milestones is set separately; see *Celebrations*.

The lamp is too heavy, or the light blinks orange or red

See *Power and load (REV A)*: the gauge on the overview tells you how heavy your lamp is, and what to do about it.

I have two lamps and mix them up

Press the crosshair at the top right (**Which one is this?**): that one lamp then blinks bright white for four seconds. Then give each lamp its own name under *Settings*, such as `zapper01` and `zapper02`. Two lamps with the same name collide on the network, and one of them cannot be found.

What clears what

ACTION	WI-FI	SETTINGS	KEY (NPUB)	ZAP STACK
Restart	kept	kept	kept	kept
Update firmware	kept	kept	kept	kept
SETUP or BOOT, 5 s	**cleared**	kept	kept	kept
Reset counters	kept	kept	kept	**cleared**
SETUP or BOOT, 10 s	**cleared**	**cleared**	**cleared**	**cleared**
Factory reset	**cleared**	**cleared**	**cleared**	**cleared**

Zap card and payments without a zap

Under **Zapping → Let others zap** is the button **Print zap card**. You get a card with the logo, "Zap this lamp" and a QR code. Choose A6, or four on an A4 to cut out yourself, and print at 100%.

Why an ordinary lightning QR does not work on its own

The lamp blinks when a zap receipt appears on nostr. A wallet that knows nothing of nostr, such as Wallet of Satoshi or Phoenix, does pay but creates no receipt. The sats arrive, but the lamp stays dark. Without a link, the card therefore carries only a QR code for nostr apps.

Link your wallet (NWC)

1. In your wallet, create a connection code for Nostr Wallet Connect. In Rizful: **Apps**, then **Get Connection Code**. Give the connection no budget: the lamp only needs to listen.
2. Paste the code under **Extras → Zapping without a nostr app** and press **Connect**.
3. A green dot and "The wallet sends payment notifications" means it works. In the Dutch interface that line is "De wallet stuurt betaalmeldingen". From now on the lamp blinks on every payment to your lightning address.
4. Print the zap card again: it now also has a QR code for any lightning wallet.

It is optional. With **Listen to the wallet: Off** the connection code is kept, but the lamp reacts only to zaps again and the zap card shows only the nostr code. Set it back to On whenever you want.

No wallet yet that sends payment notifications? Coinos

Not every wallet sends a signal over NWC when a payment comes in. **Coinos** does, and so it fits the lamp well. It is optional and can be done at any time:

1. On `coinos.io` create a free account named after the lamp, for example **zapper01**. One account per lamp: then each lamp blinks only for its own sats.
2. Enter *`name-of-your-lamp`*`@coinos.io` as the lamp's lightning address and press **Publish profile**.
3. In Coinos, create a connection for Nostr Wallet Connect, with no budget, and paste it under **Extras → Zapping without a nostr app**.
4. Green dot? Print the zap card again.

Coinos holds the sats for you. For tips that is fine; forward them to your own wallet now and then. The same explanation is in the web interface, folded out under the connection code.

> #### Use a lightning address of its own for the lamp
> With a linked wallet the lamp blinks on *every* payment to that address, including payments that have nothing to do with the lamp. A separate address for the lamp keeps it clear.

Bitcoin on-chain (optional)

Under **Extras** → **Bitcoin on-chain** the lamp can also watch the blockchain. Everything is off by default; as long as that is so, it makes no connection either.

FUNCTION	WHAT THE LAMP DOES
New block	Four calm blinks, and the status light glows orange. That is how you tell it from a zap. During quiet hours it stays still.
Payment to your address	Blinks as soon as a payment to a watched address is in the mempool, already before the first confirmation. No thank-you note on nostr, so the payment is not made public.

Address or xpub

Enter one or more addresses (up to ten), or an **xpub, ypub or zpub** from your wallet. With an xpub the lamp looks up the first unused receive address itself, follows the last used one and the next nine, and moves along as soon as one is used. A single printed QR of one address works too: that address stays watched.

> **Privacy**
> An xpub shows anyone who has it all your addresses and your balance. It stays on the lamp, but the addresses the lamp follows go to the source. With your own node, everything stays with you.

Source

By default `https://mempool.space`. If you have your own node with mempool (Umbrel, Start9, myNode), enter for example `http://umbrel.local:3006`. That is better for your privacy, and it costs the lamp no encrypted connection: with mempool.space the lamp lets one listening relay rest, to free memory.

Being found everywhere

Anyone who wants to zap your lamp must first be able to find its **profile**: that is where the lightning address is. No relay has everything, and every nostr app searches its own list of relays. That is why the lamp keeps two lists, both under **Settings → What it listens for**.

LIST	WHAT IT DOES WITH IT
Relays	It stays connected here. The zap receipt arrives here, and it publishes this list as its relay list so apps know where to send the receipt. Five at most.
Extra relays	It does not stay here. On every publish it visits them one by one to deliver only the profile and the relay list, and then disconnects again. This list may be long.

purplepag.es is the most important of the second list: it exists precisely to spread profiles and relay lists, and Damus, Amethyst and Primal fetch them from there.

> **Why not just twenty relays?**
> Every lasting connection is encrypted traffic and costs about 45 kB of memory. With three relays there is 117 kB free, with five there is 26 kB left, and a sixth connection no longer works. Three or four is the sensible number. You get known more widely with the extra relays, because those cost only one connection at a time.

Every week the lamp sends its profile round again on its own, so it does not disappear when a relay clears out old messages.

> **Publishing replaces the whole profile**
> If you edit the account in a nostr app (another picture, a display name, a banner), also enter that on the lamp. If it is not there, it is gone at the next publish.

Green and red dots

On the **Overview** tab, under *Relays*, each relay has a dot after it. Green on the upper list means the connection is open. For the publishing relays below, the dot says how the last round ended there. Green was accepted, red was refused or no answer, grey has not been tried yet. You see at a glance which relays are taking part.

A red dot on a publishing relay is rarely serious: some relays only accept messages from members or paying users. Take that one off the list, and the round is faster again too.

Do I need an nsec?

No. On first start the lamp creates its own key pair. The npub is its address, and the secret key stays in the lamp. The sats from a zap do not go through that key but straight to your lightning address.

You can import an existing nsec, but do that only if you *want* the lamp to **be** that account. On publish it replaces that account's profile with its own name and description, and every thank-you note lands on that timeline. Your personal nsec does not belong in a device that hangs on the wall.

A thank-you note on nostr

Under **Extras → Thank-you note on nostr** you can have the lamp post a short note after a zap, for example *"Thanks for 210 sats! The lamp is blinking."*The zapper's key goes along as a tag, so they get a notification. That is also the best advertisement for your lamp, and it keeps the account from looking empty.

The quiet time prevents a rain of notes when zaps come in quickly one after another. Zaps below your threshold, and zaps during quiet hours, never produce a note.

Control the lamp by nostr DM

Under **Extras → Control over nostr**, enter your own npub. From then on you can send the lamp a DM from your nostr app (Primal, Amethyst, Damus, 0xchat and others; old DMs and the newer private DMs both work) with one word:

WORD	WHAT IT DOES
party / feest	A celebration: its tune and a flash.
zap / test	A test zap.
on / aan, off / uit, auto	Always on, always off, or automatic again.
stop / stil	Stops a playing tune.
find / zoek	Blinks white, to find it.

Only DMs from that npub count. The lamp checks the signature, carries out each message once, and ignores anything older than two minutes. What it did is in the event log. Leave the field empty to switch it off.

Posting a note yourself

Under **Extras → Post a note yourself** you type a note and press **Post**. The lamp signs it with its own key, which never leaves the lamp, and sends it to the relays it listens on. A few seconds later the page shows which relays accepted it, with a link to the note as others see it.

Read it before you post: nostr has no real delete button. A deletion request exists, but not every relay or app honours it. A note is at most 2000 bytes, and the lamp posts at most one every five seconds, so a double click does not post it twice.

I do not see the lamp in Primal or in search

First look in the event log to see whether publishing was accepted. If it says *"profile accepted by purplepag.es"*, the profile can be fetched and it is not the lamp. With the lamp set to Dutch that label is *profiel* instead of *profile*.

Apps such as Primal have their own filter that keeps accounts with no history out of feeds and search results. Opening the npub directly does work. And so does zapping. What helps:

- **Follow the lamp** from your own account, and have a few people you know do the same. Those filters look at that.
- Turn the **thank-you note on nostr** on, so there are notes on the account.
- Fill in a **nostr address (NIP-05)** if you have a domain of your own. Do not invent one: an address that does not check out is shown struck through.

Taking the lamp to another home or network

The lamp remembers one Wi-Fi network. If you take it to another place, you have to tell it that new network. There are two ways, and it matters whether you do that *beforehand* or *on the spot*.

Easiest: beforehand, while you can still reach it

1. Open the lamp in your browser while it is still on the old network.
2. Go to **Settings** → **Network** and enter the name and password of the new network there straight away. Press Save and then Restart.
3. Take the lamp with you. At the new place it connects on its own; the light turns green.

This only works if you already know the new Wi-Fi password. You can prepare it at home before you leave.

On the spot: with the button on the board

1. Plug it in. The light turns **red**: the old network is not there.
2. Hold the button on the board for **5 seconds**. The light flashes blue five times.
3. It enters setup mode: the light **breathes light blue**.
4. Connect your phone to the Wi-Fi network `<name>—setup` and enter the new network. If connecting fails, use the app **ESP BLE Provisioning** and choose `PROV_<name>`; see *Setup over Bluetooth*.
5. Because the lamp has been set up before, it asks for the lamp's **current password** as a check. That stops a stranger taking it over during a move.

> **Remember the lamp password before you move**
> You need it at step 5. If you have lost it, the only way is to hold the button for 10 seconds. And then you also lose your npub and all your settings.

What you can run into at the new place

PROBLEM	WHAT TO DO
The network is 5 GHz	The lamp can only do 2.4 GHz. Many routers transmit both under the same name; then it is fine. If there is only 5 GHz, 2.4 GHz has to be turned on in the router.
The network is not in the list	Hidden networks do not appear when scanning. Type the name yourself in the field below.
Hotel Wi-Fi, or a network with a login page	Does not work. The lamp cannot accept terms or fill in a login screen. Use a phone hotspot or your own travel router.
The password has unusual characters	That is allowed, but watch for your phone automatically capitalising as you type.
You cannot reach `<name>.local`	Other routers, other rules. Look up the IP address in the new router, or set a webhook so the lamp reports its own address when it starts.
Everything is green, but no zaps	The move is fine: your nostr settings travel with it. Check that the internet works at that place.

Only the router is moving, not the lamp?

If your router gets a new name or a new password, it is the same story: set the Wi-Fi up again, preferably beforehand through the web interface. If the name stays the same and only the password changes, the lamp can no longer join and turns red.

More than one lamp

Under **Settings → Name of this lamp**, give each lamp its own number: `zapper01`, `zapper02`, `zapper03`. That name is also the web address (`http://zapper02.local`), the name of the setup network (`zapper02-setup`) and what appears at the top of the web interface.

Which number is free?

You do not have to remember which numbers you already use. Every ZAPPER announces itself on your network. On a lamp, open **Settings → ZAPPERs on this network** and press **Find ZAPPERs**: you see them all, with a link to each, and the lamp names the next free number.

A new lamp is called `zapper-` plus four characters from its chip id out of the factory, so two new lamps never get in each other's way. Once it is on your Wi-Fi, there is a button **Name this lamp zapper02**. The new name works at once.

> **A red "name taken" badge at the top?**
> Then another device has the same name. In Dutch the badge says "naam bezet". The lamp is temporarily reachable under a fallback name, such as `zapper01-2.local`, and tries every two minutes to get its own name back. Give one of the two its own number, and it is solved.

All Zappers: manage them together

From version 1.9 every lamp can manage the others on the same network, with nothing to install. Open any lamp in your browser, on your phone or your computer, and tap **All Zappers** in the menu. You see every ZAPPER with its own lamp, which follows the real one live: tap it to switch it on or off, choose Automatic, Always on or Always off, and pick the tune for a zap and for a celebration.

Tick a few lamps (or **Select all**) to act on them together, one after the other:

TAB	WHAT IT DOES
Quick	Update to the newest firmware, test zap, celebration, find me, lamp mode, restart.
Tunes	Give each its own celebration tune, play them one after another, or set the same tunes on all.
Settings	Language, sound, brightness, flashing threshold, quiet hours, milestones, looking for updates. An empty field stays as it is.
Wi-Fi	Move them to another network. Each lamp first checks that it can see that network.
Copy	Put this lamp's settings on the others. Name, Wi-Fi, password, key and tunes stay per lamp.
Backup	The settings of all selected lamps in one file, without passwords, NWC code or key.
To your phone	A QR code: scan it and the lamps appear under My Zappers in the phone app.

At the top you see the totals of all your lamps (zaps, sats, the biggest zap), and at the bottom **Latest zaps**: every zap on every lamp, with who, how many sats, the message and which lamp. Each lamp also shows its relays, Wi-Fi signal and npub. **Open** takes you to that lamp already logged in, with all its settings.

The page logs in on the other lamps with the password you used for this one, or the default Bitcoin21. A lamp with a different password asks for it once. Lamps with older firmware are shown too, with what to do: from 1.8 they update themselves under **System → Updates**.

Is everything all right?

At the top of All Zappers one line says whether all is well. If not, it names the lamp and why: no relay (zaps do not come in), output protection or too much current, a heavy load, a weak Wi-Fi signal, a warm board, a recent crash, or a lamp set to always on or off, which then does not flash on a zap. Tap the line to jump to that lamp. Each lamp shows the same as coloured chips, red for what stops zaps and orange for what is worth a look, also in the simple view.

The last 24 hours

In the nerd view each lamp shows a small line of its Wi-Fi signal over the last 24 hours, with red marks where Wi-Fi was gone. If it dropped twice or more, or no relay was connected for half an hour, the lamp says so in orange. Under the chart you see this week's sats against last week and the busiest day.

Top zappers, sats per day, and the simple view

Under the lamps, All Zappers shows **Top zappers** (from the latest zaps on all your lamps, with their nostr picture) and **Sats per day** for the last 30 days, counted by each lamp from version 1.11. The totals at the top include the current bitcoin block height. **Simple** hides the nerdy details (relays, signal, npubs, the chart); **Nerd** brings them back. **QR** on a lamp shows the code to zap it.

NIP-05 for all your lamps

Have your own website? Under the **NIP-05** tab, enter your domain and tap **Apply and publish**: each lamp gets the nostr address *name@yourdomain* and publishes its profile again. Then **Download nostr.json** and put it on your website as `/.well-known/nostr.json`, with the header `Access-Control-Allow-Origin: *`. Nostr apps then show a check mark.

The Zapper screen

For a TV, a tablet or a shop window: **Zapper screen** (in All Zappers, or in Zapper control) opens a clean page with every lamp: its lamp lighting up live, its nostr picture and name, a big QR to zap it, its lightning address, zaps and sats, the goal if one is set, and the latest zap. At the top the block height (from mempool.space) and the total. There are no settings on it: anyone can look, nobody can change anything. The back button and full screen sit in the corners and fade out after a few seconds. The QR is for the account that makes the lamp flash: its own npub, or the account it follows.

The address is `http://zapper01.local/scherm`; from All Zappers it includes your other lamps. Bookmark it on the device that shows it. The screen stays on by itself: it plays an invisible, silent video in a loop, which browsers keep the screen awake for.

⚡ **QR** in the corner adds a second code per lamp for any lightning wallet, for guests without a nostr app. It only appears for a lamp whose wallet is linked (Extras, NWC), because only then does the lamp see a payment that is not a zap.

If you want them all to react to the same zaps, set them under **Settings → What it listens for** to *Zaps to another npub* with your own npub in it. Then the whole house blinks along.

Under **System → Carry settings over** you save all settings as a file and put them back on the next lamp in one go: language, profile, how it reacts, relays, the fundraising goal, on-chain, the alert address and more. The key, the name and the password do not come along: those should be different per lamp. The wallet code (NWC) stays behind because it holds a secret, and the current calibration because it belongs to that one board.

Maintenance

Self-test

System → Run self-test checks in a second whether the chip computes correctly: signatures against the official test vectors, npub addresses, and reading invoices. All green is good. Do this after flashing a new lamp, or if zaps are refused unexpectedly.

Health

The table under **System** also shows how the lamp is doing: the chip temperature, the Wi-Fi signal over the last hour (average and lowest), why it last restarted, and its MAC address. A signal that dips below about −80 dBm explains dropouts; a lamp that keeps restarting after a voltage dip needs a better adapter.

After a power cut

You do not need to do anything. The lamp starts faster than your router; it sits red for a moment, waits until the Wi-Fi is there, and then listens again on its own.

Updates over the internet

From version 1.8 the lamp looks once a day whether a new version is out. If so, the overview says *Update available*, the event log notes it and, with alerts set up, your phone gets a message. Under **System → Updates** press **Update to …**: the lamp downloads the update, checks it and restarts by itself, in about a minute. Your settings, your key and your counters stay.

The lamp only installs an update signed with the maker's key, like a nostr note; a fake update is refused, even through a hacked network. If something goes wrong during the download, the old version keeps running. *Check now* looks right away; the daily check can be switched off on the same card.

Several lamps: under **All Zappers** you update them all in one go, one after the other (see *More than one lamp*). The newest version, the files and the earlier versions are also on `zappercontrol.netlify.app`.

Updating the firmware by hand

This goes over the network, without a cable: **System → Update firmware**, choose the update file and wait until it restarts. There are two versions, one per board: `zapper-reva-1.12.2-ota.bin` and `lamp01-1.12.2-ota.bin`, both on `zappercontrol.netlify.app`. If you pick the wrong one by mistake, the lamp refuses it and nothing changes. The `-factory.bin` files are only for a brand new board over USB. The light pulses purple meanwhile. Your settings and key stay. On that same tab, next to *Firmware*, there is a date and time: that is how you see whether the new version is really on it.

A new board, or one that no longer starts? Flash it over USB from the browser: on `zappercontrol.netlify.app`, under *Flash a new Zapper*, with Chrome or Edge on a computer and a USB-C data cable. It puts the newest complete firmware on the board, and **erases everything**, including the nostr key, so the lamp gets a new npub. Afterwards you set it up again; the password is then Bitcoin21.

> **Do not remove power during the update**
> An interrupted update is not a problem: the new version is written to the unused half of the memory and only started when it is complete. If you break it off, the old one simply keeps running.

Unplugging the USB cable (lamp01)

The cable is only needed to program the lamp the first time. After that everything goes over the network. Do provide power: an ordinary USB charger is enough, or put the 5 V supply that already feeds the lamp onto the `5V` and `GND` pins of the board as well. Then you have one supply for the whole thing.

Starting over

System → Factory reset clears everything: Wi-Fi, password, settings and the nostr key. The lamp then gets a *new* npub, so anyone who saved the old one zaps an account nobody reads any more.

From a computer

Zapper control is a program for Windows, Mac and Linux that does what All Zappers does, also for lamps from before 1.9, with the Zapper screen and a QR per lamp. Download it from `zappercontrol.netlify.app` and double-click it; nothing else to install. The first time, Windows and macOS warn about an unknown maker; the download page shows the one click to get past that.

Why http, and from outside your home

The lamp speaks plain http on your own network, like WLED, Tasmota, Shelly and the Bitaxe. A certificate for https would only give a browser warning on a device at home, and costs memory the relays need. Your password never travels in readable form, sessions are one-time codes, the lamp only answers pages from your own network, and updates are signed like nostr. To manage your lamps from elsewhere, use a VPN into your home network such as Tailscale or WireGuard; do not open the lamp to the internet with port forwarding.

On the lamp's serial port (USB, 115200 baud) a few commands help with testing:

```
zap 21000     a simulated zap: tests the goal and the milestones
tune cancan   play a tune
tunes         list the tunes
stop          stop the tune
```

A simulated zap counts in the Zap stack, but posts nothing on nostr and sends no webhook.

Technical data

	ZAPPER REV A	LAMP01
Chip	ESP32-C6-WROOM-1-N8: RISC-V, 160 MHz, 8 MB flash	Waveshare ESP32-C6-Zero, 8 MB flash
Power	USB-C, 5 V, 3 A max (no USB PD)	external 5 V supply
Lamp output	USB-A via TPS2559, on/off; limit 2.48 to 2.8 A	GPIO4 to a MOSFET, PWM 5 kHz, 8 bits
Measurement	output voltage and current (0.5 V/A), calibratable	none
Status light	RGB LED, GPIO18/19/20, active low	WS2812 on GPIO8, RGB order
Buttons	SETUP GPIO21, ACTION GPIO22, BOOT, RESET	BOOT GPIO9
Buzzer	GPIO10, LEDC PWM; tunes between C6 and C8 (1 to 4.2 kHz)	none

BOTH BOARDS	
Web interface	HTTP, port 80
Bluetooth	BLE 5 (NimBLE), only during setup mode; provisioning per Espressif's protocomm, security 1 (x25519 + PIN)
Serial port	115200 baud, USB-CDC
Relays	Four at a time recommended, five is the maximum (about 45 kB of heap per connection); they must speak TLS 1.2. Publishing relays: up to twelve, one connection at a time
Zaps	NIP-57, kind 9735; signature and sender are checked
Alerts	HTTP POST with text (ntfy headers Title, Priority, Tags), fault and overload at most once per 10 minutes

About ZAPPER

Designed and built by **JaccoElektro**, offered to **BitPopArt** (bitpopart.com).

ZAPPER	npub14cgegu5krm5gacdn8eqatfpmmwd6dlm8sx3sjrdjyav0ltuvl0ns6yx52u
JaccoElektro	npub1vxehyetdargg2vc4hjvxlm65y2w6fqgzyz0hukcskeh6yamdkmwqvn87da
BitPopArt	npub1gwa27rpgum8mr9d30msg8cv7kwj2lhav2nvmdwh3wqnsa5vnudxqlta2sz

Built on the nostr standards (NIP-01, 05, 19, 24, 57, 65), BIP340, uBitcoin, ArduinoJson, arduinoWebSockets, QRCode and Arduino-ESP32/ESP-IDF from Espressif. The idea is inspired by nostr-zap-lamp from LNbits. The firmware and tools in this repository are MIT; arduinoWebSockets is LGPL-2.1.
